Operating System Principles

Module – 3

Deadlock

Deadlock Avoidance

Deadlock Avoidance

- Deadlock-prevention algorithms prevent deadlocks by limiting how requests can be made.
- The limits ensure that at least one of the necessary conditions for deadlock cannot occur.
- Possible side effects of preventing deadlocks by this method, however, are low device utilization and reduced system throughput.

Deadlock Avoidance

- An alternative method for avoiding deadlocks is to require additional information about how resources are to be requested.
- With this knowledge of the complete sequence of requests and releases for each thread, the system can decide for each request whether or not the thread should wait in order to avoid a possible future deadlock.

Deadlock Avoidance

- The simplest and most useful model requires that each thread declare the maximum number of resources of each type that it may need.
- Given this a priori information, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state.

Deadlock Avoidance

- A deadlock avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular-wait condition can never exist.
- The resource-allocation ***state*** is defined by the number of available and allocated resources and the maximum demands of the threads.

Deadlock Avoidance

Safe State

- A state is *safe* if the system can allocate resources to each thread in some order and still avoid a deadlock.
- A system is in a safe state only if there exists a *safe sequence*.
- If no safe sequence exists, then the system state is said to be *unsafe*.

Deadlock Avoidance

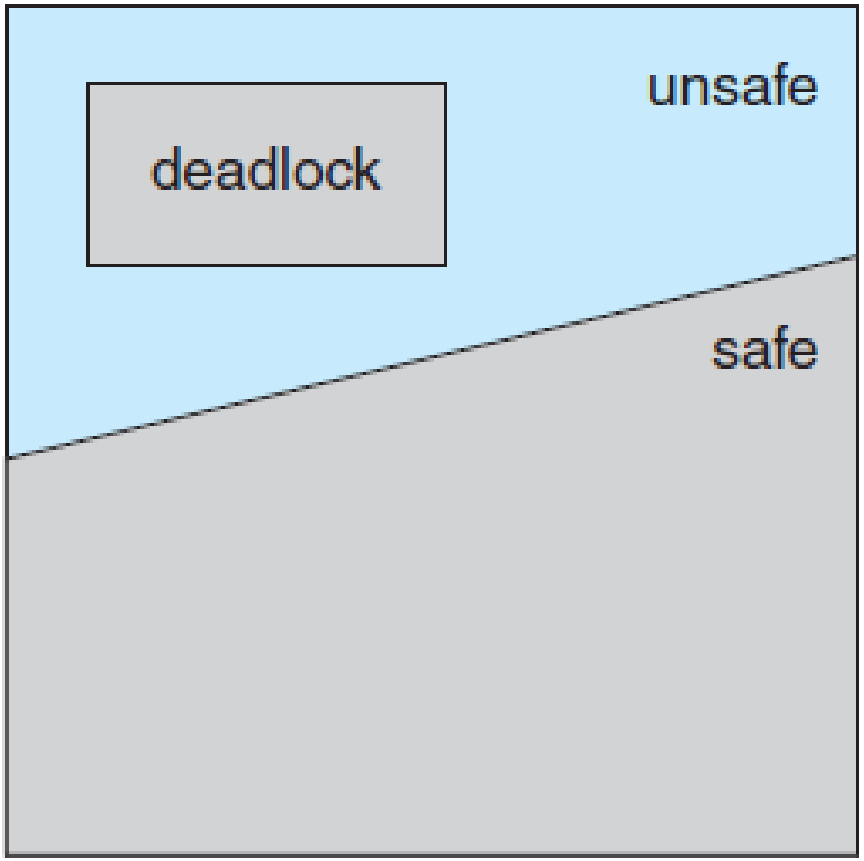


Figure Safe, unsafe, and deadlocked state spaces

Deadlock Avoidance

An example: 12 resources, 3 threads

	Maximum Needs	Current Needs
T_0	10	5
T_1	4	2
T_2	9	2

Deadlock Avoidance

An example: 12 resources, 3 threads

	Maximum Needs	Current Needs
T_0	10	5
T_1	4	2
T_2	9	2

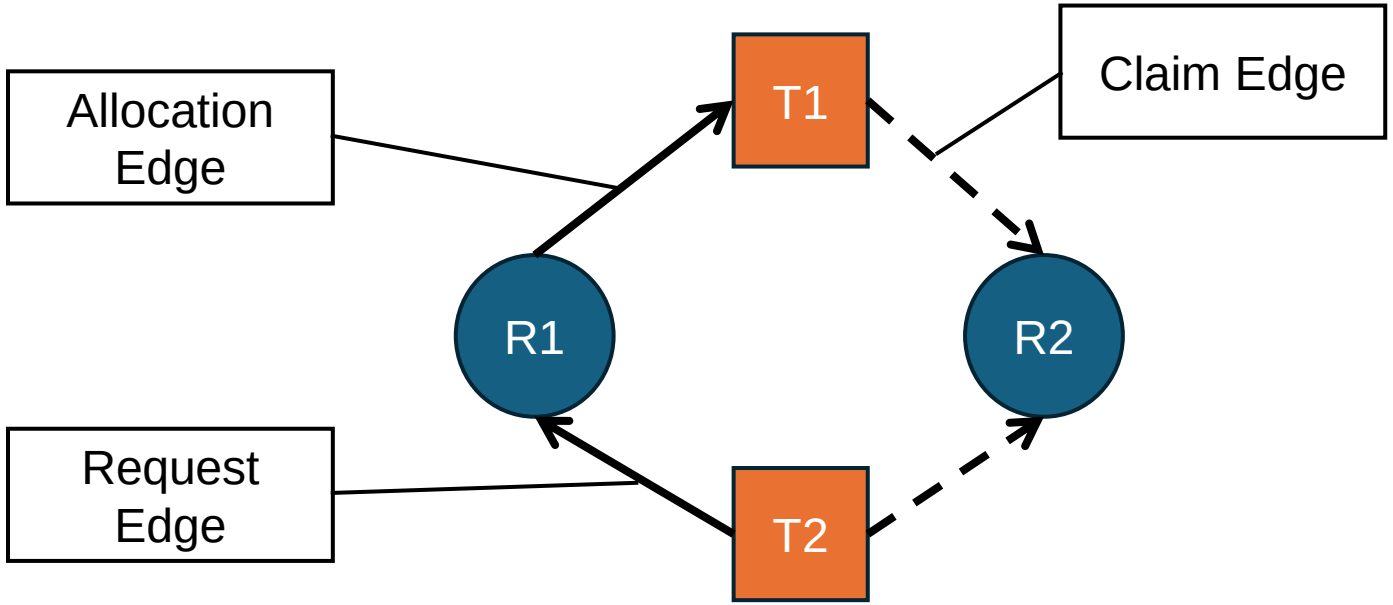
The sequence <*T*1, *T*0, *T*2> satisfies the safety condition.

Define avoidance algorithms that ensure that the system will never deadlock.

What happens if T_2 request one more resource at t_2 ?

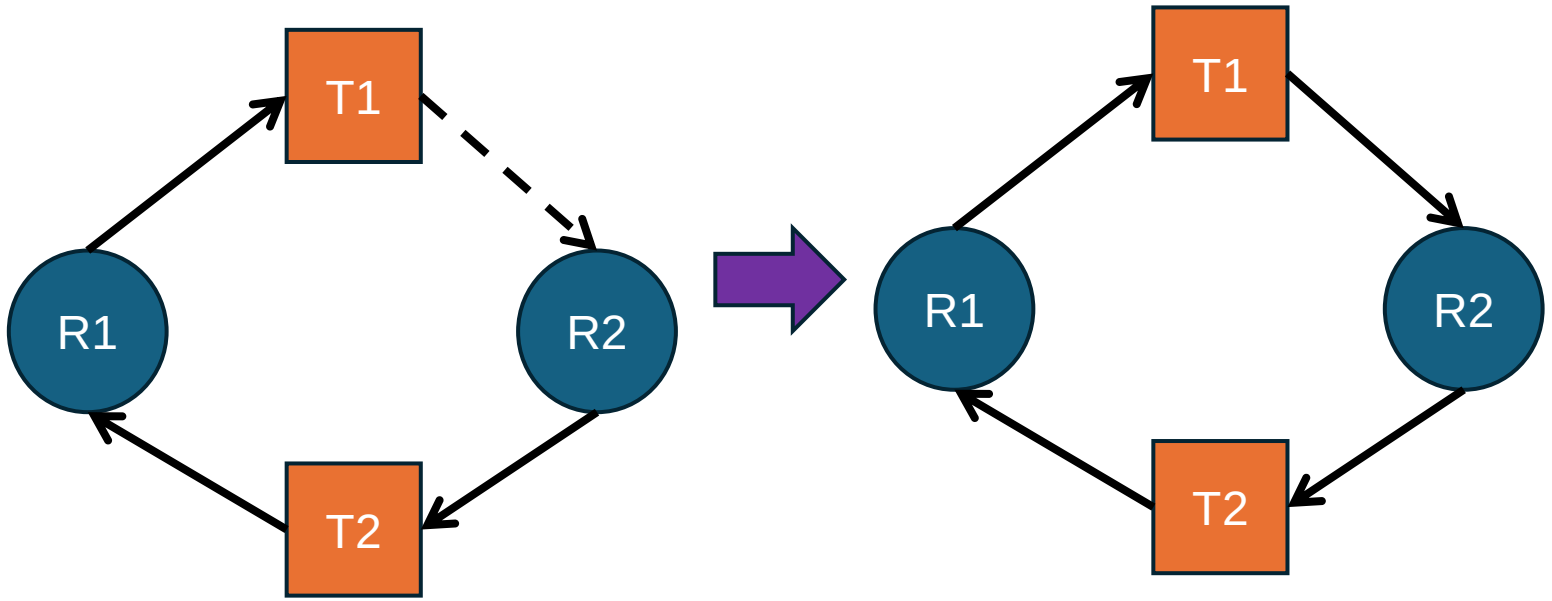
Deadlock Avoidance - Resource-Allocation-Graph Algorithm

- Only Single Instance of Resources exist in system
  - Method Applied is Resource Allocation Graph.
  - Example:



Deadlock Avoidance

- Converting claim edge to Request Edge and then to Allocation Edge.
- Example: Cycle forms and then Deadlock is identified.



Deadlock Avoidance

- The request can be granted only if converting the request edge $T_i \rightarrow R_j$ to an assignment edge $R_j \rightarrow T_i$ does not result in the formation of a cycle in the resource-allocation graph.
- A **cycle-detection algorithm** is used
- It requires an order of **n^2 operations**, where n is the number of threads in the system.

Deadlock Avoidance - Banker's Algorithm

- The resource-allocation-graph algorithm is not applicable to a resource allocation system with multiple instances of each resource type.
- When a new thread enters the system, it must declare the maximum number of instances of each resource type that it may need.

Deadlock Avoidance - Banker's Algorithm

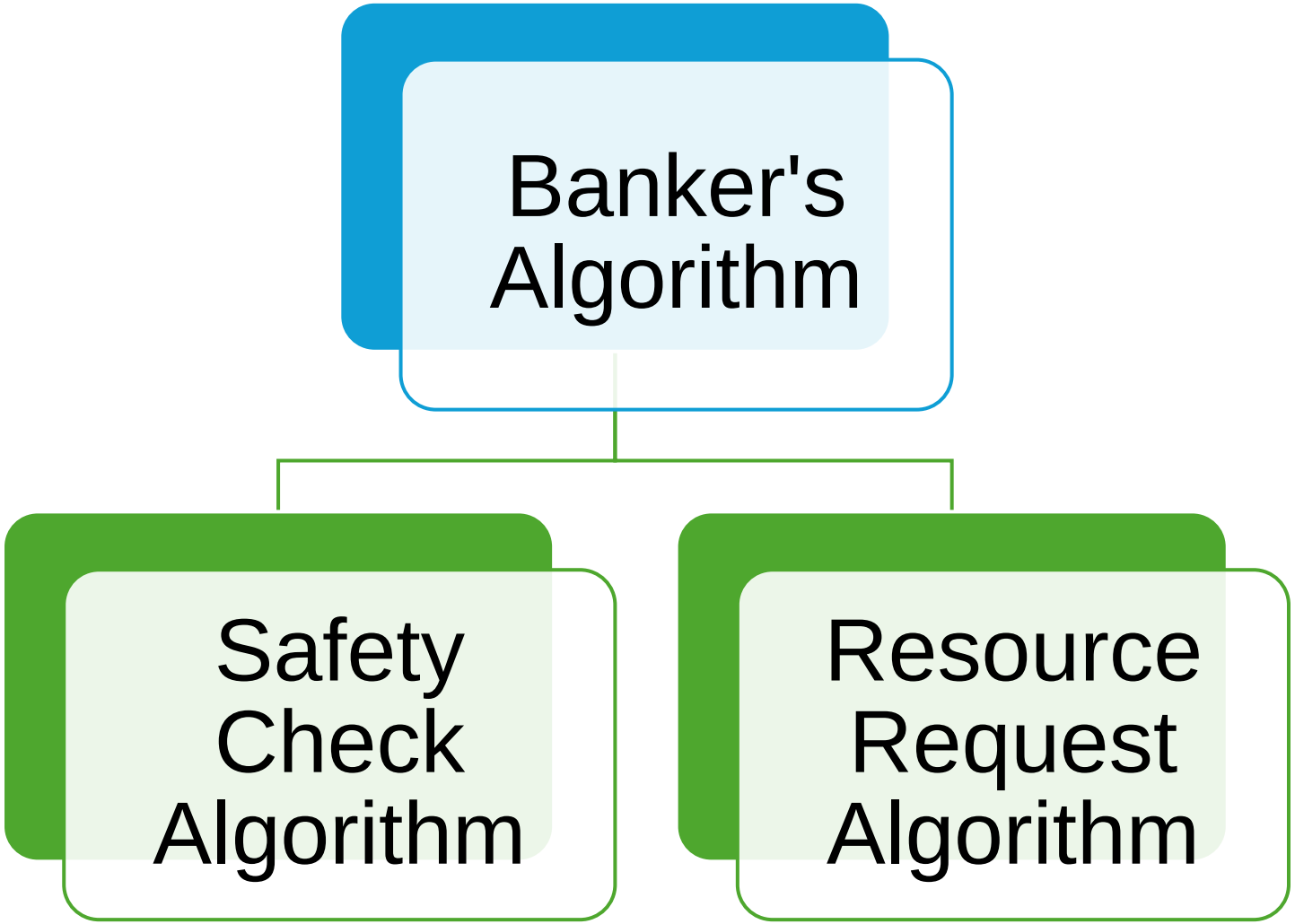
- This number may not exceed the total number of resources in the system.
- When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state.
- If it will, the resources are allocated; otherwise, the thread must wait until some other thread releases enough resources.

Deadlock Avoidance

Data structures used to implement Banker's algorithm

- **Available**- the number of available resources of each type
- **Max**- the maximum demand of each thread
- **Allocation**- the number of resources of each type currently allocated to each thread
- **Need**- the remaining resource need of each thread.

Deadlock Avoidance



Deadlock – Management - Avoidance

- **Safety algorithm** checks for the safe state of the system. If system is in safe state with any of the resource allocation permutation, then deadlock can be avoided.

- **Resource request algorithm** checks if the request can be safely granted or not to the process requesting for resources.

Deadlock – Avoidance – Bankers Algorithm

Example 1

Process	RESOURCE ALLOCATION			MAXIMUM RESOURCE		
	R1	R2	R3	R1	R2	R3
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

- Consider the current system where resources R1, R2 and R3 respectively are consumed by Processes P0 to P4. The total number of Instances of each resource in the system is given as **R1=10, R2=5 and R3=7.**
- Apply Banker's algorithm to check whether the system is in safe state and also find the safe sequence.

Deadlock – Avoidance – Bankers Algo

Process	RESOURCE ALLOCATION			MAXIMUM RESOURCE		
	R1	R2	R3	R1	R2	R3
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

- Calculate the current Available resources
- R1 = 10 – 7(2+3+2) = 3
- R2 = 5 – 2(1+1) = 3
- R3 = 7 – 5(2+1+2) = 2

Deadlock – Avoidance – Bankers Algo

- Calculate the **Resource Need** of every Process

Process	RESOURCE ALLOCATION			MAXIMUM RESOURCE			NEED RESOURCES		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

Deadlock – Avoidance – Bankers Algo

- Compare All Available Resources with All Need Resources, if All Need is less than or equal to All Available then we can allocate else check with next process

Process	RESOURCE ALLOCATION			MAXIMUM RESOURCE			NEED RESOURCES			Is R1, R2 & R3 can be allocated?
	R1	R2	R3	R1	R2	R3	R1	R2	R3	Available - 3 3 2
P0	0	1	0	7	5	3	7	4	3	No
P1	2	0	0	3	2	2	1	2	2	Yes
P2	3	0	2	9	0	2	6	0	0	
P3	2	1	1	2	2	2	0	1	1	
P4	0	0	2	4	3	3	4	3	1	

Deadlock – Avoidance – Bankers Algo

Process	RESOURCE ALLOCATION			MAXIMUM RESOURCE			NEED			Allocation Possible	(After Allocation & Process termination) Resource Available Status		
	R1	R2	R3	R1	R2	R3	R1	R2	R3		R1	R2	R3
P0	0	1	0	7	5	3	7	4	3	No			
P1	2	0	0	3	2	2	1	2	2	Yes	**5**	**3**	**2**
P2	3	0	2	9	0	2	6	0	0	No			
P3	2	1	1	2	2	2	0	1	1				
P4	0	0	2	4	3	3	4	3	1				

Process Sequence – P1 –

Deadlock – Avoidance – Bankers Algo

Process	RESOURCE ALLOCATION			MAXIMUM RESOURCE			**NEED**			**Allocation Possible**	**(After Allocation & Process termination) Resource Available Status**		
	R1	R2	R3	R1	R2	R3	**R1**	**R2**	**R3**		**R1**	**R2**	**R3**
P0	0	1	0	7	5	3	7	4	3	No			
P1	2	0	0	3	2	2	1	2	2	Yes	**5**	**3**	**2**
P2	3	0	2	9	0	2	6	0	0	No			
P3	2	1	1	2	2	2	0	1	1	Yes	**7**	**4**	**3**
P4	0	0	2	4	3	3	4	3	1				

Process Sequence – P1 – P3 –

Deadlock – Avoidance – Bankers Algo

Process	RESOURCE ALLOCATION			MAXIMUM RESOURCE			NEED			Allocation Possible	(After Allocation & Process termination) Resource Available Status		
	R1	R2	R3	R1	R2	R3	R1	R2	R3		R1	R2	R3
P0	0	1	0	7	5	3	7	4	3	No			
P1	2	0	0	3	2	2	1	2	2	Yes	5	3	2
P2	3	0	2	9	0	2	6	0	0	No			
P3	2	1	1	2	2	2	0	1	1	Yes	7	4	3
P4	0	0	2	4	3	3	4	3	1	Yes	7	4	5

Process Sequence – P1 – P3 – P4 –

Deadlock – Avoidance – Bankers Algo

Process	RESOURCE ALLOCATION			MAXIMUM RESOURCE			**NEED**			**Allocation Possible**	**(After Allocation & Process termination) Resource Available Status**		
	R1	R2	R3	R1	R2	R3	**R1**	**R2**	**R3**		**R1**	**R2**	**R3**
P0	0	1	0	7	5	3	7	4	3	Yes	**7**	**5**	**5**
P1	2	0	0	3	2	2	1	2	2	Yes	**5**	**3**	**2**
P2	3	0	2	9	0	2	6	0	0	Yes	**10**	**5**	**7**
P3	2	1	1	2	2	2	0	1	1	Yes	**7**	**4**	**3**
P4	0	0	2	4	3	3	4	3	1	Yes	**7**	**4**	**5**

Process Sequence – P1 – P3 – P4 – P0 – P2

Deadlock – Avoidance – Bankers Algo

Process	RESOURCE ALLOCATION			MAXIMUM RESOURCE			**NEED**			**Allocation Possible**	**(After Allocation & Process termination) Resource Available Status**		
	R1	R2	R3	R1	R2	R3	**R1**	**R2**	**R3**		**R1**	**R2**	**R3**
P0	0	1	0	7	5	3	7	4	3	Yes	**7**	**5**	**5**
P1	2	0	0	3	2	2	1	2	2	Yes	**5**	**3**	**2**
P2	3	0	2	9	0	2	6	0	0	Yes	**10**	**5**	**7**
P3	2	1	1	2	2	2	0	1	1	Yes	**7**	**4**	**3**
P4	0	0	2	4	3	3	4	3	1	Yes	**7**	**4**	**5**

Process Sequence – P1 – P3 – P4 – P0 – P2

Bankers Algorithm

Resource Request Algorithm

If Request <= Need

{

If Request <= Available

{

Available = Available – Request

Allocation = Allocation + request

Need = Need – Request

}

}

Safety Check Algorithm

Work = Available

Finish[i] = false for i=0 … n-1

Find such i that

Finish[i] = FALSE && Need <= Work

Work = Work + Allocation

Finish[i] = TRUE

If (Finish[i] = TRUE for all i = 0 … n-1)

Then system is in Safe state.

Deadlock – Avoidance – Bankers Algo – Example 2

- Consider the process table with number of processes that contains:
- Allocation field (for showing the number of resources of type: R1, R2 and R3 allocated to each process in the table),
- Max field (for showing the maximum number of resources of type: A, B, and C that can be allocated to each process).
- Available resources are: **R1=3, R2=2, R3=1**

Process	Allocation			MAX		
	R1	R2	R3	R1	R2	R3
P0	1	1	2	5	4	4
P1	2	1	2	4	3	3
P2	3	0	1	9	1	3
P3	0	2	0	8	6	4
P4	1	1	2	2	2	3

Deadlock – Management - Avoidance

- Step 1: Calculate the Process's resource Need.

Process	ALLOCATION			MAX			**NEED**		
	R1	R2	R3	R1	R2	R3	**R1**	**R2**	**R3**
P0	1	1	2	5	4	4	**4**	**3**	**2**
P1	2	1	2	4	3	3	**2**	**2**	**1**
P2	3	0	1	9	1	3	**6**	**1**	**2**
P3	0	2	0	8	6	4	**8**	**4**	**4**
P4	1	1	2	2	2	3	**1**	**1**	**1**

Deadlock – Management - Avoidance

- Step 2: Check for Safe State

Process	ALLOCATION			MAX			**NEED**			**Allocation Possible**
	R1	R2	R3	R1	R2	R3	**R1**	**R2**	**R3**	**Check the Need with available resource instances. Helps to check Safe state or not.**
P0	1	1	2	5	4	4	**4**	**3**	**2**	**No**
P1	2	1	2	4	3	3	**2**	**2**	**1**	**Yes**
P2	3	0	1	9	1	3	**6**	**1**	**2**	**No**
P3	0	2	0	8	6	4	**8**	**4**	**4**	**No**
P4	1	1	2	2	2	3	**1**	**1**	**1**	**Yes**

- Available
  - R1 = 3, R2 = 2, R3 =1

Deadlock – Management - Avoidance

- Step 3: If Safe State exists then find the process safe sequence execution.

Process	ALLOCATION			MAX			**NEED**			**Allocation Possible**	**New Available (After Allocation & Process termination)**		
	R1	R2	R3	R1	R2	R3	**R1**	**R2**	**R3**		**R1**	**R2**	**R3**
P0	1	1	2	5	4	4	**4**	**3**	**2**	**No**	**7**	**5**	**7**
P1	2	1	2	4	3	3	**2**	**2**	**1**	**Yes**	**5**	**3**	**3**
P2	3	0	1	9	1	3	**6**	**1**	**2**	**No**	**10**	**5**	**8**
P3	0	2	0	8	6	4	**8**	**4**	**4**	**No**	**10**	**7**	**8**
P4	1	1	2	2	2	3	**1**	**1**	**1**	**Yes**	**6**	**4**	**5**

- Process Sequence – P1 – P4 – P0 – P2 – P3

Reference

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts”, 2018, 10th Edition, Wiley, United States.